Full name: ABDELRAHIM Ghenaim
Class: Group 02

How to Work with Big Data Files

1. Introduction

Goal. The goal of this experiment is to learn how to work with very large files and to see how they affect the computer memory (RAM). This is a typical *big data* problem: when a file is bigger than the available RAM, the usual tools stop working.

We test three techniques, from the simplest to the most advanced:

1. **Reading the whole file at once** with pandas `read_csv`.
2. **Reading the file in chunks, one after another**, with `read_csv(chunksize=100000)`.
3. **Using Dask**, which splits the file into parts and processes them in parallel.

Dataset. We used the *Flight Prices* dataset from Kaggle.[1] Its size is 31 GB when uncompressed. It contains about 6 million rows and 27 columns. A file of this size cannot easily fit in the memory of a normal computer, so it is a good case study for this experiment.

Laptop specifications. We used 16GB ddr4 of Ram with Depian 13 lunix System.

2. Traditional Method: pandas `read_csv`

How it works. The function `pd.read_csv()` reads the *whole* file and builds one big DataFrame in RAM. To do this, pandas must keep every row and every column in memory at the same time. In practice, the DataFrame often needs *more* memory than the file size on disk, because text columns are stored as Python objects.

```
import pandas as pd
# loads everything in RAM
# Also we can use the "usecols" parameter to make a supdataset
# to see the results in short time.
df = pd.read_csv(dataset_path)
```

Result. The memory usage grew quickly until all the RAM (16 GB) was used. The system became very slow, and the program finally stopped with an **Out Of Memory (OOM)** error. The data was never fully loaded, so this method failed.

3. First Solution: Reading in Chunks

How it works. With the parameter `chunksize`, pandas does not return one DataFrame. It returns an iterator that gives small DataFrames of 100 000 rows each. We process one chunk,

keep only the partial result, and then the chunk is released from memory before the next one is read.

```
import pandas as pd

total_fare_count = 0
total_fare_sum = 0
# Also we can use the "usecols" parameter to make a supdataset
# to see the results in short time.
for chunk in pd.read_csv(dataset_path, chunksize=100_000):
       fares = chunk["totalFare"]
       total_fare_sum += fares.sum()
       total_fare_count += fares.count()

global_mean = total_fare_sum / total_fare_count

print("mean: ",global_mean)
print("total: ", total_fare_sum)
```

Advantages

- Very low and stable RAM usage.
- Works on any computer.
- Simple, with the usual pandas syntax.

Disadvantages

- Chunks are read one by one (sequential), so it is slower.

4. Last Solution: Dask

How Dask reads the data. Dask splits the CSV file into many partitions (blocks of a fixed size in bytes). Each partition is a normal pandas DataFrame, but Dask does not load them immediately. `dd.read_csv()` only reads the structure of the file and prepares a plan. This is called *lazy evaluation*.

How Dask computes. Each operation (filter, mean, groupby, …) is added to a *task graph*. Nothing runs until we call `.compute()`. Then Dask runs the tasks on all CPU cores in parallel, works on a few partitions at a time, and combines the partial results into the final answer.

```
import dask.dataframe as dd

cols = ["totalFare", "totalTravelDistance", "travelDuration"]

dask_df = dd.read_csv(
    dataset_path, blocksize="256MB" ,
    dtype={"totalTravelDistance": "float64"}, usecols=cols[:2]
    )

total_fare_mean = dask_df["totalFare"].mean()
total_fare_sum = dask_df["totalFare"].sum()
result1 , result2 = dask.compute(total_fare_mean, total_fare_sum)

print("dask (mean): ",result1)
print("dask (sum): ",result2)
```

5. Comparison of the Three Experiments

Method	RAM usage	Execution time
pandas `read_csv` (full file)	All the RAM (16 GB)	Very long
pandas with chunks (100 000 rows)	Constant, 157 MB	Medium: 236 s
Dask	4.3 GB	Very fast: 71 s

Table 1: Comparison of the three ways of reading the Flight Prices dataset.

6. Conclusion

Working with big data mainly affects the RAM in a negative way: when a file is larger than the memory, the classic method of loading everything at once fails. For this reason, we need advanced solutions that keep the memory under control.

In our experiments, reading the file in chunks kept the RAM very low and stable (157 MB), but it took 236 seconds. Dask used more memory (4.3 GB), but it finished in only 71 seconds, which is about 3.3 times faster. Therefore, we conclude that **Dask was the best solution** for this type of challenge.

Notes

[1]Kaggle, *Flight Prices* dataset: https://www.kaggle.com/datasets/dilwong/flightprices